TLS Challenge Report

Overview

The goal of this challenge was to secure a server implementation with TLS by building a CLI tool that generates digital certificates in .pem format. The tool was implemented in Python 3 using the cryptography library, and the certificates were tested with the provided Python server implementation.

Tool: certgen.py

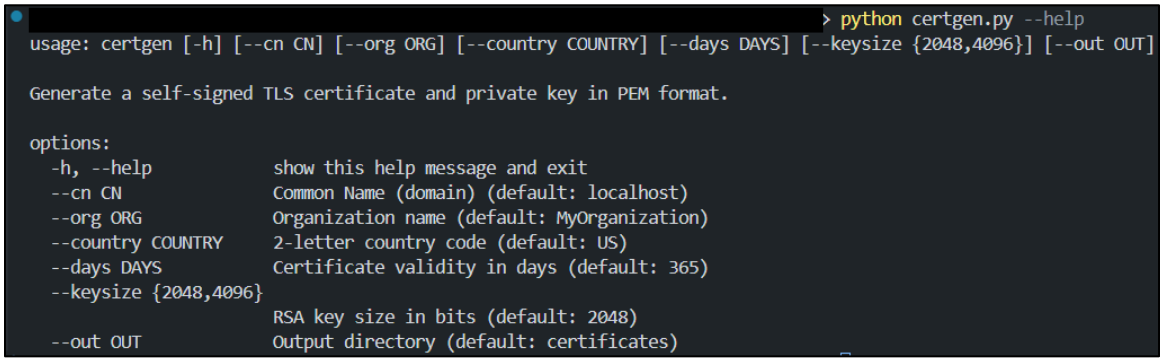
The CLI tool certgen.py generates a self-signed X.509 certificate and an RSA private key, saving them as cert.pem and key.pem inside a certificates/ directory.

Dependencies:

- Python 3
- cryptography library (pip install cryptography)

Example command of execution:

python certgen.py --cn localhost --org "MyOrganization" --country US



Output files:

- certificates/cert.pem — the digital certificate
- certificates/key.pem — the private key

Certificate Details

FIELD	VALUE
SUBJECT	CN=localhost, O=MyOrganization, C=US
SERIAL	392460556288010967306490971014920533614006916185
VALID FROM	2026-05-13
VALID UNTIL	2027-05-13
ALGORITHM	SHA-256 with RSA (2048-bit)
FORMAT	X.509 v3, PEM encoded

```
--------------------------------------------------
  Certificate Generated Successfully
--------------------------------------------------
  Subject  : CN=localhost,O=MyOrganization,C=US
  Serial   : 256899247396350227211526712226258468726698252815
  Valid    : 2026-05-20 → 2027-05-20
  Key file : certificates\key.pem
  Cert file: certificates\cert.pem
--------------------------------------------------
```

Server Testing

- The certificates were tested using the provided Python server implementation (server.py). The server was started with:
- python server.py
- Upon navigating to https://localhost:4000, the browser displayed the server's directory listing over a secure HTTPS connection, confirming that TLS is functioning correctly.

